

Linguagem C++

Módulo 3 · Ponteiros

Conceito, operadores, aplicação com vetores e funções, e alocação dinâmica de memória

Prof. Antonio Marcos Selmini

● O que veremos neste módulo

- 1 Memória e endereços**
Como as variáveis vivem na memória e o operador &
- 2 O conceito de ponteiro**
Declaração, o operador *, nullptr e ponteiros para ponteiros
- 3 Ponteiros e vetores**
Aritmética de ponteiros e a equivalência $v[i] \equiv *(v + i)$
- 4 Ponteiros e funções**
Passagem por ponteiro, retorno de ponteiros e const
- 5 Alocação dinâmica**
new, delete, vetores e matrizes dinâmicos
- 6 Armadilhas e boas práticas**
Vazamentos, ponteiros pendentes e a relação com `std::vector`

1

Memória e endereços

Onde as variáveis ficam e como localizá-las

● Como a memória é organizada

- A memória principal é uma longa sequência de bytes, cada um com um número que o identifica: o endereço.
- Ao declarar uma variável, o programa reserva um bloco de bytes consecutivos para ela (4 para int, 8 para double...).
- O endereço da variável é o endereço do primeiro byte desse bloco.
- Normalmente não nos preocupamos com endereços: usamos o nome da variável. Os ponteiros mudam isso.

```
int a = 10;    double x = 2.5;  
char c = 'A';
```

a	10	1000 (4 B)
x	2.5	1004 (8 B)
c	'A'	1012 (1 B)

Endereços simplificados; na prática são números grandes, exibidos em hexadecimal.

● O operador de endereço: &

```
int a = 10;
double x = 2.5;

cout << a << endl;      // 10
cout << &a << endl;     // 0x7ffd5c3e
cout << &x << endl;     // 0x7ffd5c40

cout << sizeof(a);      // 4
cout << sizeof(&a);     // 8 (64 bits)
```

C++

Atenção `int& r = a` (referência) e `&a` (endereço) usam o mesmo símbolo com sentidos diferentes: após um tipo, referência; antes de uma variável, endereço.

- Colocado antes de uma variável, & devolve o endereço dela: “o endereço de a”.
- O resultado é exibido em hexadecimal (prefixo 0x) e varia a cada execução.
- Todo endereço ocupa o mesmo espaço (8 bytes em sistemas de 64 bits), qualquer que seja o tipo da variável.
- Um endereço é um valor como outro qualquer — e pode ser guardado em uma variável. Essa variável é o ponteiro.

2

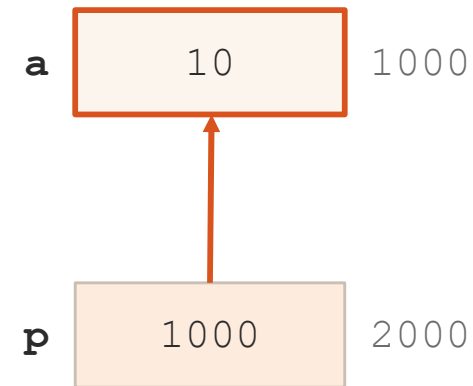
O conceito de ponteiro

Uma variável que guarda o endereço de outra

● O que é um ponteiro?

- Um ponteiro é uma variável cujo valor é o endereço de outra variável.
- Diz-se que o ponteiro “aponta para” a variável cujo endereço ele guarda.
- Declara-se com um asterisco após o tipo apontado: `int* p`; lê-se “p é um ponteiro para int”.
- O ponteiro também ocupa memória e tem o próprio endereço.

```
int a = 10;  
int* p = &a;
```



p guarda 1000, o endereço de a: p aponta para a

Convenção Nos diagramas, caixas em pêssego são ponteiros e a seta indica para onde apontam.

● Declarando ponteiros

```
int* p;           // ponteiro para int
double* q;        // ponteiro para double
char* c;          // ponteiro para char

int *p2;          // mesmo que int* p2

int a = 5;
double x = 1.5;
p = &a;           // OK: tipos combinam
q = &x;           // OK
p = &x;           // ERRO: int* ≠ double*

int* r, s;        // r é ponteiro,
                  // s é um int comum!
```

- O tipo antes do * é o tipo da variável apontada; o ponteiro só pode receber endereços desse tipo.
- O espaço ao redor do * é indiferente para o compilador: `int* p` e `int *p` são iguais.
- Em declarações múltiplas, o * vale apenas para o nome imediatamente seguinte. Declare um ponteiro por linha para evitar confusão.
- Um ponteiro declarado sem valor contém lixo: aponta para um lugar qualquer.

● O operador de desreferência: *

```
int a = 10;
int* p = &a;

cout << p;      // 1000 (endereço)
cout << *p;     // 10 (valor apontado)

*p = 25;        // escreve em a
cout << a;      // 25

a = 7;
cout << *p;     // 7
```

C++

- Colocado antes de um ponteiro, * acessa o conteúdo do endereço apontado: “o valor apontado por p”.
- *p pode ser lido e escrito. Escrever em *p é escrever na variável apontada.
- Lê-se: “desreferenciar p” ou “acessar o conteúdo de p”.
- p e *p são coisas distintas: p é um endereço; *p é o valor que está nesse endereço.

Lembre p é uma variável separada de a; * é a ponte entre elas.

● Os dois papéis de & e * — resumo

Símbolo	Na declaração (após um tipo)	Na expressão (antes de uma variável)
&	Declara uma referência <code>int& r = a;</code>	Endereço de <code>&a</code> devolve o endereço de <code>a</code>
*	Declara um ponteiro <code>int* p = &a;</code>	Conteúdo apontado por <code>*p</code> devolve o valor em <code>a</code>

Regra prática

Ao lado de um tipo: declaração. Ao lado de uma variável em uma expressão: operador.

& e * se cancelam

`*(&a)` é o próprio `a`; e `&(*p)` é o próprio `p`.

● O ponteiro nulo: nullptr

```
int* p = nullptr;    // não aponta
                     // para nada

if (p == nullptr) {
    cout << "Sem destino";
}

if (p) {              // se não nulo
    cout << *p;
}

cout << *p;          // ERRO em execução:
                     // acesso inválido
```

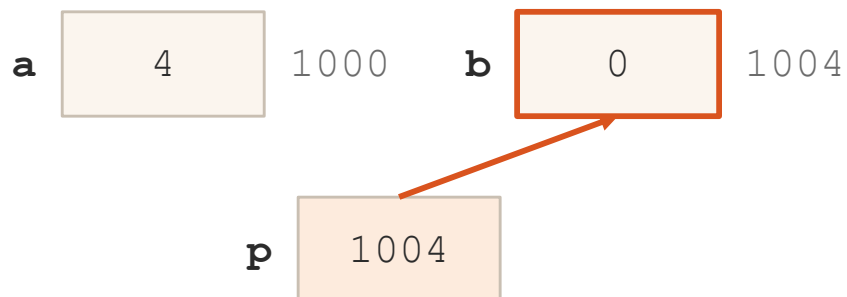
- nullptr (C++11) representa “nenhum endereço”. É o valor recomendado para inicializar todo ponteiro que ainda não tem destino.
- Código antigo usa NULL ou 0 com o mesmo propósito; prefira nullptr.
- Em condições, um ponteiro nulo equivale a false e um não nulo a true.
- Desreferenciar um ponteiro nulo encerra o programa com erro (segmentation fault). Verifique antes de usar *.

● Acompanhando passo a passo

```
int a = 3, b = 8;  
int* p = &a;      // (1)  
*p = *p + 1;      // (2)  
p = &b;           // (3)  
*p = 0;           // (4)  
cout << a << " " << b;  // 4 0
```

C++

Estado final:



Após	a	b	p aponta para	*p
(1)	3	8	a	3
(2)	4	8	a	4
(3)	4	8	b	8
(4)	4	0	b	0

Observe Reatribuir p (p = &b) muda o destino do ponteiro; atribuir a *p muda o valor da variável apontada. São operações independentes.

● Por que o ponteiro precisa de tipo?

- Todos os ponteiros têm o mesmo tamanho: guardam apenas um endereço.
- O tipo diz ao compilador como interpretar os bytes a partir daquele endereço: quantos ler e como convertê-los em valor.
- Um `int*` lê 4 bytes como inteiro; um `double*` lê 8 bytes como real; um `char*` lê 1 byte como caractere.
- O tipo também define quanto o ponteiro avança em cada passo da aritmética (próxima seção).

```
int a = 65;
int* pi = &a;
char* pc = (char*) &a;

cout << *pi;    // 65
cout << *pc;    // 'A' (1º byte)

cout << sizeof(pi); // 8
cout << sizeof(pc); // 8
cout << sizeof(*pi); // 4
cout << sizeof(*pc); // 1
```

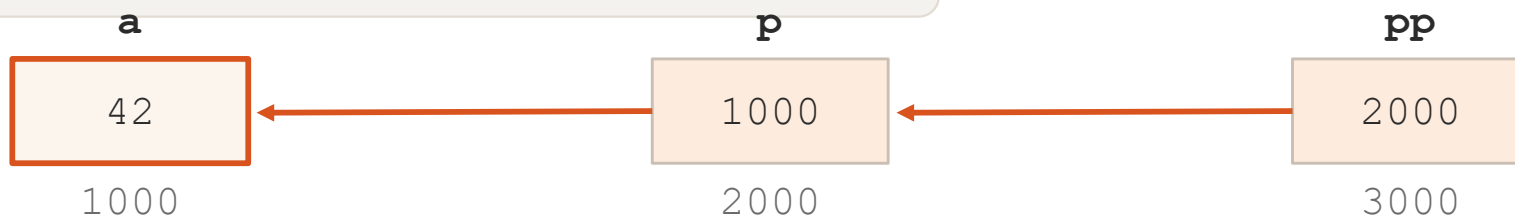
C++

Nota A conversão `(char*)` força a reinterpretação dos bytes; é um recurso avançado, mostrado apenas para ilustrar o papel do tipo.

● Ponteiro para ponteiro

```
int a = 10;  
int* p = &a;  
int** pp = &p;    // aponta para p  
  
cout << *pp;       // 1000 (valor de p)  
cout << **pp;      // 10  (valor de a)  
  
**pp = 42;         // altera a  
cout << a;         // 42
```

C++



- Como o ponteiro é uma variável, ele também tem endereço — e outro ponteiro pode guardá-lo.
- `int**` é “ponteiro para ponteiro para `int`”. Cada `*` a mais é um nível de indireção.
- Aparece em matrizes dinâmicas (`int**`) e em funções que precisam alterar um ponteiro.

● Ponteiros × referências

	Referência <code>int& r = a;</code>	Ponteiro <code>int* p = &a;</code>
O que é	Um apelido para a	Variável com o endereço de a
Pode ser nula?	Não — sempre se refere a algo	Sim (nullptr)
Pode mudar de alvo?	Não — fixa após a inicialização	Sim — atribuindo outro endereço
Acesso ao valor	Direto: r	Com desreferência: *p
Aritmética	Não existe	Sim (p + 1, p++)
Uso típico	Parâmetros de função	Vetores, memória dinâmica

Recomendação Quando uma referência resolve, prefira-a: é mais simples e segura. Use ponteiros quando precisar de nulo, troca de alvo, aritmética ou alocação dinâmica.



3

Ponteiros e vetores

A relação íntima entre o nome de um vetor e um endereço

● O nome do vetor é um endereço

```
int v[4] = {10, 20, 30, 40};

int* p = v;      // igual a &v[0]

cout << v;       // 1000
cout << &v[0];   // 1000
cout << p;       // 1000
cout << *p;      // 10
cout << *v;      // 10
```

C++



- Em expressões, o nome de um vetor estático é convertido no endereço do primeiro elemento.
- Por isso um ponteiro do tipo do elemento pode receber o vetor diretamente, sem &.
- `*v` e `v[0]` são o mesmo valor.

● Aritmética de ponteiros

```
int v[4] = {10, 20, 30, 40};
int* p = v;           // 1000

p + 1;                // 1004
p + 3;                // 1012
*(p + 2);             // 30

p++;                  // p = 1004
cout << *p;           // 20

int* q = &v[3];
cout << q - p;        // 2 (elementos)
```

p+0	p+1	p+2	p+3
10	20	30	40

- Somar 1 a um ponteiro avança um elemento, não um byte: para int*, 4 bytes; para double*, 8.
- É por isso que o ponteiro tem tipo: o compilador sabe o tamanho do passo.
- Subtrair dois ponteiros do mesmo vetor dá a distância em elementos.
- Operações válidas: p + n, p - n, p++, p--, p - q, comparações (<, ==...). Somar dois ponteiros não faz sentido.

● A equivalência fundamental: $v[i] \equiv *(v + i)$

```
int v[4] = {10, 20, 30, 40};  
int* p = v;
```

```
v[2]          // 30  
*(v + 2)      // 30  
p[2]          // 30  
*(p + 2)      // 30
```

```
p[2] = 99;    // altera v[2]
```

C++

- O operador `[ ]` é apenas uma abreviação: `v[i]` significa “vá *i* elementos adiante e desreferencie”.
- Como consequência, `[ ]` funciona com qualquer ponteiro, não apenas com vetores.
- É essa regra que faz os índices começarem em 0: `v[0]` é `*(v + 0)`, o primeiro elemento.

Diferença `v` é um nome fixo (não se pode fazer `v++` nem `v = p`). `p` é uma variável e pode ser reapontado. Fora isso, comportam-se igual.

● Percorrendo um vetor com ponteiros

```
const int N = 5;
int v[N] = {3, 1, 4, 1, 5};

// com índice
for (int i = 0; i < N; i++)
    cout << v[i] << " ";
// com ponteiro
for (int* p = v; p < v + N; p++)
    cout << *p << " ";
// soma com ponteiro
int soma = 0;
for (int* p = v; p != v + N; p++)
    soma += *p;
```

- O ponteiro começa em v (primeiro elemento) e avança um elemento por iteração.
- $v + N$ é o endereço logo após o último elemento: é válido para comparação, mas nunca deve ser desreferenciado.
- $p < v + N$ e $p != v + N$ são as formas usuais de condição.
- Esse padrão é a base dos iteradores da biblioteca padrão (`begin()` e `end()`).

● Ponteiros e matrizes estáticas

```
int m[2][3] = {{1,2,3},{4,5,6}};  
  
int* p = &m[0][0]; // 1º elemento  
  
cout << *(p + 4); // 5 (m[1][1])  
  
// percorre os 6 elementos  
for (int* q = p; q < p + 6; q++)  
    cout << *q << " ";  
// 1 2 3 4 5 6
```

- A matriz é armazenada de forma contígua, linha após linha (ordem row-major).
- O elemento $m[i][j]$ está na posição $i \times \text{colunas} + j$ a partir do início.
- Um ponteiro para o primeiro elemento percorre a matriz inteira como se fosse um vetor.



● Strings no estilo C: char*

```
char s[] = "Ola";    // 4 bytes:
                    // O l a \0
const char* p = s;

while (*p != '\0') {
    cout << *p;
    p++;
}

int tam = 0;
for (const char* q = s; *q; q++)
    tam++;           // tam = 3
```

'O'	'l'	'a'	'\0'
-----	-----	-----	------

- Antes de `std::string`, textos eram vetores de `char` terminados pelo caractere nulo `'\0'`.
- Um `char*` aponta para o primeiro caractere; percorre-se até encontrar `'\0'`.
- Como `'\0'` vale zero, a condição `*q` é falsa exatamente no fim.
- Ainda aparecem em `argv` do `main`, em APIs de sistema e em código legado; em código novo, prefira `std::string`.

4

Ponteiros e funções

Compartilhando variáveis entre funções por meio de endereços

● Passagem de parâmetros por ponteiro

```
void dobrar(int* n) {  
    *n = *n * 2;    // altera x  
}  
  
int main() {  
    int x = 5;  
    dobrar(&x);    // passa endereço  
    cout << x;    // 10  
}
```

- A função recebe o endereço da variável; com *n acessa e altera a variável original.
- Na chamada, é obrigatório passar um endereço: dobrar(&x), não dobrar(x).
- Este é o mecanismo que a linguagem C usa; em C++ a referência (int& n) faz o mesmo com sintaxe mais limpa.

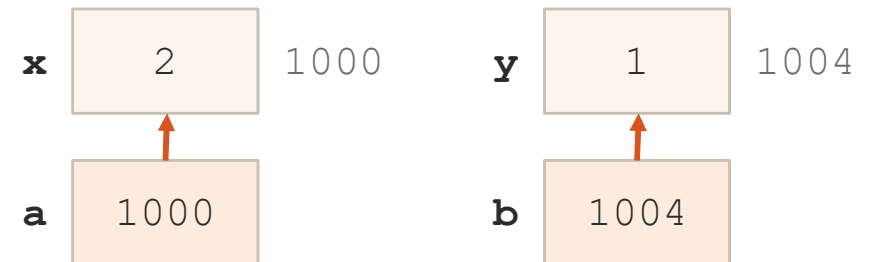
Por valor	Por referência	Por ponteiro
<code>void f(int n)</code>	<code>void f(int& n)</code>	<code>void f(int* n)</code>
<code>f(x);</code> — copia	<code>f(x);</code> — original	<code>f(&x);</code> — original

● Exemplo clássico: trocar dois valores

```
void trocar(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
  
int main() {  
    int x = 1, y = 2;  
    trocar(&x, &y);  
    cout << x << " " << y;    // 2 1  
}
```

Dentro de trocar a e b são cópias dos endereços;
apontam para as variáveis do main.

- Sem ponteiros (ou referências), trocar(x, y) receberia cópias e a troca não sairia da função.
- temp guarda o valor apontado por a antes que ele seja sobrescrito.
- Cada *a e *b lê ou escreve diretamente em x e y do main.



● Revisitando: vetores como parâmetros

```
// as três assinaturas são idênticas
void mostrar(int v[], int n);
void mostrar(int v[10], int n);
void mostrar(int* v, int n);

void zerar(int* v, int n) {
    for (int i = 0; i < n; i++)
        v[i] = 0;          // ou *(v+i) = 0
}

int d[3] = {1, 2, 3};
zerar(d, 3);               // d vira o endereço
sizeof(d);                 // 12 no main
                           // 8 dentro de zerar!
```

- No Módulo 2 vimos que um vetor passado a uma função “não é copiado”. Agora sabemos por quê: a função recebe apenas o endereço do primeiro elemento.
- `int v[]` em um parâmetro é apenas outra grafia de `int* v`.
- Por isso o tamanho se perde e precisa ser passado à parte — `sizeof` no parâmetro mede o ponteiro, não o vetor.
- Para proteger o vetor, use `const int* v`.

● Funções que retornam ponteiros

```
// OK: devolve o endereço do maior
int* maior(int* a, int* b) {
    return (*a > *b) ? a : b;
}

int x = 3, y = 9;
int* m = maior(&x, &y);
*m = 0;           // zera y

// PERIGO: variável local
int* errado() {
    int local = 42;
    return &local; // local morre
}                  // ao retornar
```

- Uma função pode devolver um endereço, permitindo que o chamador acesse ou altere a variável apontada.
- Regra de ouro: só devolva endereços de variáveis que continuam existindo após o retorno — parâmetros recebidos por ponteiro, variáveis globais ou memória alocada dinamicamente.
- Variáveis locais são destruídas ao fim da função; o endereço devolvido passa a ser um ponteiro pendente (dangling pointer).

● const e ponteiros: o que fica protegido?

```
int a = 1, b = 2;

const int* p1 = &a; // valor fixo
*p1 = 5;           // ERRO
p1 = &b;           // OK

int* const p2 = &a; // endereço fixo
*p2 = 5;           // OK
p2 = &b;           // ERRO

const int* const p3 = &a;
*p3 = 5;           // ERRO
p3 = &b;           // ERRO
```

Declaração	Altera *p	Altera p
int* p	Sim	Sim
const int* p	Não	Sim
int* const p	Sim	Não
const int* const p	Não	Não

Como ler Leia da direita para a esquerda: const int* p → “p é ponteiro para int constante”; int* const p → “p é ponteiro constante para int”.

5

Alocação dinâmica

Criando variáveis durante a execução com new e delete

● Duas regiões de memória: pilha e heap

Pilha (stack)

Guarda variáveis locais e parâmetros. A memória é reservada ao entrar na função e liberada automaticamente ao sair. Rápida, mas de tamanho limitado e fixo em compilação.

Heap (memória dinâmica)

Região grande, requisitada explicitamente com `new` e devolvida com `delete`. O tamanho pode ser decidido em execução e a memória sobrevive ao fim da função que a criou — enquanto não for liberada.

- Como a memória dinâmica não tem nome, o único acesso a ela é pelo endereço devolvido por `new` — ou seja, por um ponteiro.
- Quem pede memória com `new` é responsável por devolvê-la com `delete`.

● new e delete para uma variável

```
int* p = new int;          // aloca
*p = 10;
cout << *p;               // 10

int* q = new int(7);       // já com valor
double* d = new double{2.5};

delete p;                  // libera
delete q;
delete d;
p = nullptr;              // boa prática
```

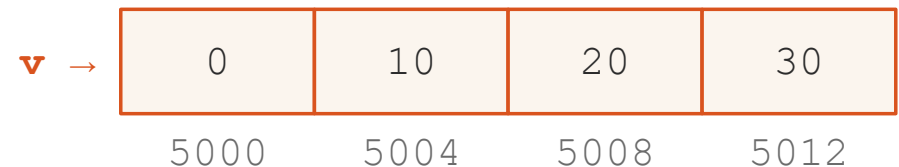


- new tipo reserva espaço na heap para um valor do tipo e devolve o endereço.
- O valor inicial pode ser dado entre parênteses ou chaves.
- delete p devolve a memória apontada por p ao sistema. A variável p continua existindo, mas seu conteúdo passa a ser inválido.
- Atribuir nullptr após delete impede o uso acidental do endereço antigo.

● Vetores dinâmicos: new[] e delete[]

```
int n;  
cin >> n;                // ex.: 4  
  
int* v = new int[n];      // n ints  
int* z = new int[n]();    // zerados  
  
for (int i = 0; i < n; i++)  
    v[i] = i * 10;        // índice  
  
cout << v[2];             // 20  
cout << *(v + 3);         // 30  
  
delete[] v;               // com [ ]!  
delete[] z;  
v = nullptr;
```

- new tipo[n] aloca n elementos contíguos e devolve o endereço do primeiro; n pode ser uma variável.
- O ponteiro devolvido é usado exatamente como um vetor: índices e aritmética funcionam.
- Memória alocada com new[] deve ser liberada com delete[] — o par deve combinar.
- O tamanho não é armazenado: guarde n em uma variável.



● Exemplo completo: média de n valores

```
int main() {  
    int n;  
    cout << "Quantos valores? ";  
    cin >> n;  
    double* v = new double[n];  
    double soma = 0;  
    for (int i = 0; i < n; i++) {  
        cin >> v[i];  
        soma += v[i];  
    }  
    cout << "Média: " << soma / n;  
    delete[] v;  
    return 0;  
}
```

- O tamanho do vetor só é conhecido após a leitura de n — exatamente o caso que os vetores estáticos não resolvem.
- Após o uso, delete[] devolve a memória.
- Todo caminho de saída da função deve liberar a memória; se um return antecipado pular o delete, ocorre vazamento.

Compare No Módulo 2, vector<double> v(n) fez o mesmo sem delete. Agora vemos o mecanismo que ele usa por dentro.

● Matrizes dinâmicas com int**

```
int lin = 2, col = 3;

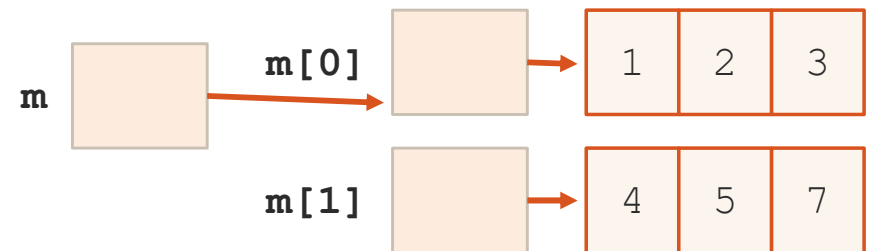
// 1) vetor de ponteiros (linhas)
int** m = new int*[lin];

// 2) cada linha: um vetor de int
for (int i = 0; i < lin; i++)
    m[i] = new int[col];

m[1][2] = 7;    // uso normal

// liberar na ordem inversa
for (int i = 0; i < lin; i++)
    delete[] m[i];
delete[] m;
```

- m aponta para um vetor de ponteiros, um por linha; cada m[i] aponta para um vetor de col inteiros.
- m[i][j] funciona porque m[i] é um int* e [j] indexa a partir dele.
- Libere primeiro as linhas e depois o vetor de ponteiros — a ordem inversa da alocação.



6

Armadilhas e boas práticas

Os erros mais comuns e como evitá-los

● Quatro erros clássicos

Vazamento de memória (leak)

`int* p = new int[100];` e nenhum `delete[]`. A memória fica presa até o programa terminar. Em laços, esgota o sistema.

Ponteiro pendente (dangling)

Usar `p` depois de `delete p`, ou guardar o endereço de uma variável local que já foi destruída. Comportamento imprevisível.

Delete duplo

`delete p; delete p;` — a segunda liberação corrompe a heap. Atribua `nullptr` após o primeiro `delete` (`delete nullptr` é seguro).

Par errado

`new int[n]` com `delete p`, ou `new int` com `delete[] p`. Compila, mas é comportamento indefinido. Combine sempre `new/delete` e `new[]/delete[]`.

● Checklist de boas práticas

- ✓ Inicialize todo ponteiro: com um endereço válido ou com `nullptr`. Nunca deixe com lixo.
- ✓ Verifique se o ponteiro não é nulo antes de desreferenciar.
- ✓ Para cada `new`, exatamente um `delete`; para cada `new[ ]`, um `delete[ ]`.
- ✓ Após `delete`, atribua `nullptr` ao ponteiro.
- ✓ Nunca devolva nem guarde o endereço de uma variável local de outra função.
- ✓ Prefira referências a ponteiros em parâmetros quando não precisar de nulo.
- ✓ Prefira `std::vector` e `std::string` à alocação manual: fazem `new/delete` por você.

● Fechando o ciclo: ponteiros e std::vector

```
// o que vector faz por dentro
// (versão simplificada)
double* dados = new double[n];
int tamanho = n;
// ... push_back: aloca maior,
//      copia, libera o antigo ...
delete[] dados;    // no fim da vida
```

C++

- std::vector guarda internamente um ponteiro para um bloco alocado com new[] e cuida do delete[] automaticamente.
- Ao usar vector, você usa ponteiros sem escrever nenhum — com segurança.
- O C++ moderno também oferece ponteiros inteligentes (unique_ptr, shared_ptr, em <memory>) que liberam a memória sozinhos.

Então por que estudar ponteiros? Para entender o que acontece por baixo, para trabalhar com estruturas de dados (listas, árvores), com APIs em C e com código legado — e para depurar quando algo dá errado.

● Exercícios propostos

- 1 Declare `int a = 5` e `int* p = &a`; imprima `a`, `&a`, `p`, `*p` e `&p`, e explique cada valor.
- 2 Escreva `void incrementar(int* n)` que soma 1 à variável apontada; teste com uma variável do `main`.
- 3 Implemente `int soma(int* v, int n)` percorrendo o vetor apenas com aritmética de ponteiros (sem `[ ]`).
- 4 Escreva `int tamanho(const char* s)` que devolve o comprimento de uma string estilo C.
- 5 Leia `n`, aloque um vetor com `new[ ]`, preencha-o, mostre-o em ordem inversa e libere a memória.
- 6 Aloque uma matriz `lin × col` com `int**`, preencha com `i + j`, imprima e libere na ordem correta.

Resumo do módulo

- Toda variável tem um endereço (&a); um ponteiro guarda um endereço; *p acessa o valor apontado; nullptr indica “nenhum”.
- O nome de um vetor é o endereço do primeiro elemento; v[i] equivale a *(v + i).
- Por ponteiro, funções alteram variáveis do chamador; vetores são passados como endereço.
- new/delete e new[]/delete[] criam e liberam memória na heap; cada new exige seu delete.
- Prefira referências, std::vector e ponteiros inteligentes; ponteiros crus quando necessário.

Referência: STROUSTRUP, B. Programming: Principles and Practice Using C++. 3. ed. Addison-Wesley, 2024.